

Blockchain-Based Evidence Preservation System

Project Documentation

1. Introduction

The Blockchain-Based Evidence Preservation System is a web-based law enforcement application designed to securely manage, track, and verify physical evidence collected during criminal investigations in Tamil Nadu. It allows authorized officers - Police, Forensic Staff, and Evidence Room Officers - to log, amend, and audit evidence records while protecting data integrity using blockchain technology.

Each evidence entry is linked using SHA-256 cryptographic hashes, making any unauthorized modification instantly detectable. The public can independently verify the chain of custody through a dedicated Public Ledger Lookup portal, while internal access is restricted to authorized personnel through a hidden, role-based authentication gateway.

2. Software Used

Software	Purpose
Python 3.x	Backend Language
Flask	Web Framework
SQLite	Database Management
SQLAlchemy	Database ORM
Flask-Login	User Session and Authentication
Werkzeug	Password Hashing and File Utilities
HTML / CSS	Frontend Structure and Styling
Bootstrap 5	Responsive UI Framework
JavaScript	Client-Side Interactivity
SHA-256	Blockchain Hashing Algorithm
Font Awesome	Icon Library

3. Project Modules

1. Public Ledger Lookup (Landing Page)

- Open to citizens, lawyers, and third-party auditors
- Search by Case ID, keyword, Tamil Nadu district, or date range
- Displays chain of custody, block history, and cryptographic hashes
- No authentication required

2. Hidden Authentication Module

- Login page accessible only via secret URL: /secure-gateway-7741
- Not linked anywhere publicly - only known to authorized personnel
- Role-based access: Admin, Forensic Staff, Evidence Room Officer, Police
- Quick-fill credential panel for authorized users

3. Registry Dashboard

- All logged evidence in a sortable table
- Shows Case ID, thumbnail, description, district, measurements, event type, custodian
- Filter by keyword, Tamil Nadu district dropdown, and date-from/date-to range
- Role-based action buttons (Log Evidence, Correct Evidence)

4. Evidence Logging Module (INITIAL Block)

- Register new evidence with auto-generated Case ID (EV-XXXXXXXX)
- Fields: description, quantity, size/weight, seizure district
- Upload image or document attachment (JPG, PNG, PDF, DOCX, etc.)
- Entry is cryptographically signed and appended to the blockchain ledger

5. Evidence Correction Module (CORRECTION Block)

- Submit a court-order-backed amendment to an existing evidence record
- Requires mandatory Court Order Reference Number
- Appends a new correction block; original block remains permanently unchanged
- Optionally replace the attached file

6. Blockchain Module

- Creates Genesis Block on system initialization
- Every block contains: Evidence ID, Block Type, Description, Count, Size, District,
- Officer Details, Court Order, Correction Notes, Attached File, Timestamp, Hashes
- Chained: each block's previous_hash points to the prior block's hash

7. Blockchain Verification (Admin Audit Board)

- Recalculates SHA-256 hash for every block in sequence
- Verifies each block's previous_hash matches the actual prior hash
- Detects tampering - any modification invalidates the hash chain
- Admin can simulate tampering and restore integrity for demonstration

8. Evidence Chain of Custody History

- Full sequential block-by-block audit trail for any evidence item
- Displays every INITIAL and CORRECTION event with timestamps, officer details,
- district, court order references, and cryptographic hashes

4. Source Code

4.1 Application Configuration (app.py)

```
import os, datetime, hashlib, uuid
from flask import Flask, render_template, redirect, url_for, request, flash
from flask_sqlalchemy import SQLAlchemy
from flask_login import LoginManager, UserMixin, login_user, logout_user, login_required,
current_user
from werkzeug.security import generate_password_hash, check_password_hash

app = Flask(__name__)
app.config['SECRET_KEY'] = 'blockchain-evidence-preservation-secret-key-12345'
app.config['SQLALCHEMY_DATABASE_URI'] = 'sqlite:///evidence_system.db'
app.config['SQLALCHEMY_TRACK_MODIFICATIONS'] = False

SECRET_LOGIN_PATH = '/secure-gateway-7741'
db = SQLAlchemy(app)
login_manager = LoginManager(app)
```

```
login_manager.login_view = 'secret_login'
```

4.2 User Model (app.py)

```
class User(UserMixin, db.Model):
    id = db.Column(db.Integer, primary_key=True)
    username = db.Column(db.String(50), unique=True, nullable=False)
    password_hash = db.Column(db.String(255), nullable=False)
    role = db.Column(db.String(30), nullable=False)
    fullname = db.Column(db.String(100), nullable=False)
    badge_number = db.Column(db.String(30), nullable=False)

    def set_password(self, password):
        self.password_hash = generate_password_hash(password)

    def check_password(self, password):
        return check_password_hash(self.password_hash, password)
```

4.3 Evidence Block Model (app.py)

```
class EvidenceBlock(db.Model):
    id = db.Column(db.Integer, primary_key=True)
    evidence_id = db.Column(db.String(50), nullable=False)
    block_type = db.Column(db.String(30), nullable=False)
    item_description = db.Column(db.Text, nullable=False)
    count = db.Column(db.Integer, nullable=False)
    size = db.Column(db.String(50), nullable=False)
    district = db.Column(db.String(100), nullable=True)
    timestamp = db.Column(db.DateTime, default=datetime.datetime.utcnow)
    officer_details = db.Column(db.String(150), nullable=False)
    court_order_number = db.Column(db.String(100), nullable=True)
    correction_notes = db.Column(db.Text, nullable=True)
    evidence_file = db.Column(db.String(255), nullable=True)
    evidence_file_type = db.Column(db.String(50), nullable=True)
    previous_hash = db.Column(db.String(64), nullable=False)
    hash = db.Column(db.String(64), nullable=False)
```

4.4 SHA-256 Hash Generation (app.py)

```
def calculate_block_hash(self):
    ts_str = self.timestamp.strftime('%Y-%m-%d %H:%M:%S')
    data_string = (
        str(self.evidence_id) + "|" + str(self.block_type) + "|" +
        str(self.item_description) + "|" + str(self.count) + "|" +
        str(self.size) + "|" + str(self.district or "") + "|" + ts_str + "|" +
        str(self.officer_details) + "|" + str(self.court_order_number or "") + "|" +
        str(self.correction_notes or "") + "|" + str(self.previous_hash)
    )
    return hashlib.sha256(data_string.encode('utf-8')).hexdigest()
```

4.5 Genesis Block Creation (app.py)

```
def create_genesis_block():
    genesis = EvidenceBlock(
        evidence_id = "SYSTEM-GENESIS",
        block_type = "SYSTEM",
        item_description = "Genesis block of the evidence preservation ledger.",
        count=0, size="0", district="",
        timestamp = datetime.datetime(2026, 1, 1, 0, 0, 0),
        officer_details = "SYSTEM",
        previous_hash = "0"
    )
    genesis.hash = genesis.calculate_block_hash()
    db.session.add(genesis)
```

```
db.session.commit()
return genesis
```

4.6 Appending a New Evidence Block (app.py)

```
def add_new_block(evidence_id, block_type, item_description, count,
size, officer, district=None, court_order=None,
notes=None, evidence_file=None, evidence_file_type=None):
    latest = get_latest_block()
    officer_info = officer.fullname + " (" + officer.badge_number + ") - " + officer.role
    new_block = EvidenceBlock(
        evidence_id=evidence_id, block_type=block_type,
        item_description=item_description, count=count, size=size,
        district=district, timestamp=datetime.datetime.utcnow(),
        officer_details=officer_info, court_order_number=court_order,
        correction_notes=notes, evidence_file=evidence_file,
        evidence_file_type=evidence_file_type, previous_hash=latest.hash
    )
    new_block.hash = new_block.calculate_block_hash()
    db.session.add(new_block)
    db.session.commit()
    return new_block
```

4.7 Evidence Logging Route (app.py)

```
@app.route('/evidence/log', methods=['GET', 'POST'])
@login_required
def log_evidence():
    if current_user.role not in ['Police', 'Forensic Staff', 'Evidence Room Officer']:
        flash('Unauthorized access.', 'danger')
        return redirect(url_for('dashboard'))
    if request.method == 'POST':
        item_description = request.form.get('item_description')
        count = request.form.get('count', type=int)
        size = request.form.get('size')
        district = request.form.get('district', '').strip()
        evidence_file = request.files.get('evidence_file')
        filename, file_type = save_uploaded_file(evidence_file)
        evidence_id = 'EV-' + uuid.uuid4().hex[:8].upper()
        add_new_block(evidence_id=evidence_id, block_type="INITIAL",
            item_description=item_description, count=count, size=size,
            district=district or None, officer=current_user,
            evidence_file=filename, evidence_file_type=file_type)
        flash('Evidence ' + evidence_id + ' logged.', 'success')
        return redirect(url_for('dashboard'))
    return render_template('log_evidence.html', tn_districts=TN_DISTRICTS)
```

4.8 Blockchain Verification (app.py)

```
def verify_blockchain_integrity():
    blocks = EvidenceBlock.query.order_by(EvidenceBlock.id.asc()).all()
    is_compromised = False
    verification_log = []
    for i, block in enumerate(blocks):
        calculated_hash = block.calculate_block_hash()
        hash_valid = (block.hash == calculated_hash)
        link_valid = (block.previous_hash == blocks[i-1].hash) if i > 0 else True
        block_valid = hash_valid and link_valid
        if not block_valid:
            is_compromised = True
        verification_log.append({
            'id': block.id, 'evidence_id': block.evidence_id,
            'hash_valid': hash_valid, 'link_valid': link_valid,
            'block_valid': block_valid, 'db_hash': block.hash,
```

```
'calculated_hash': calculated_hash
}))
return {'is_secure': not is_compromised, 'log': verification_log}
```

4.9 Public Search with District and Date Filters (app.py)

```
@app.route('/search')
def public_search():
    query = request.args.get('query', '').strip()
    district_f = request.args.get('district', '').strip()
    date_from = request.args.get('date_from', '').strip()
    date_to = request.args.get('date_to', '').strip()
    has_any_filter = bool(query or district_f or date_from or date_to)
    results = []
    if has_any_filter:
        all_blocks = EvidenceBlock.query.order_by(EvidenceBlock.timestamp.desc()).all()
        results = build_evidence_list(all_blocks, query, district_f, date_from, date_to)
    return render_template('public_search.html',
        query=query, district_filter=district_f, date_from=date_from,
        date_to=date_to, results=results, tn_districts=TN_DISTRICTS,
        has_any_filter=has_any_filter)
```

4.10 Dashboard Route (app.py)

```
@app.route('/dashboard')
@login_required
def dashboard():
    search_query = request.args.get('search', '').strip()
    district_f = request.args.get('district', '').strip()
    date_from = request.args.get('date_from', '').strip()
    date_to = request.args.get('date_to', '').strip()
    all_blocks = EvidenceBlock.query.order_by(EvidenceBlock.timestamp.desc()).all()
    evidence_list = build_evidence_list(all_blocks, search_query,
        district_f, date_from, date_to)
    return render_template('dashboard.html',
        evidence_list=evidence_list, search_query=search_query,
        district_filter=district_f, date_from=date_from,
        date_to=date_to, tn_districts=TN_DISTRICTS)
```

5. Output Screenshots

[Attach screenshots of the following pages from the running application]

#	Page	Description
1	Public Search Portal	Homepage with keyword, district, and date-range search
2	Hidden Login Page	/secure-gateway-7741 with quick-fill credential panel
3	Registry Dashboard	Evidence table with district badges and filter controls
4	Log Evidence Form	New evidence entry with district dropdown and file upload
5	Evidence History	Sequential blockchain audit trail for a single case
6	Admin Audit Board	Blockchain integrity verification with hash comparison
7	Request Correction	Court-order-backed amendment form

6. Conclusion

The Blockchain-Based Evidence Preservation System demonstrates how blockchain technology can significantly improve the security, integrity, and transparency of physical evidence management in law enforcement. By chaining every evidence log entry using SHA-256 cryptographic hashes, the system ensures that any unauthorized modification is immediately detectable and permanently recorded.

The project combines Flask, SQLite, SQLAlchemy, and Flask-Login to deliver a robust, role-segregated web platform that meets the practical needs of Tamil Nadu law enforcement agencies. The hidden authentication gateway ensures that only authorized officers can interact with evidence records, while the public-facing ledger search empowers citizens, lawyers, and auditors to independently verify case histories without requiring an account.

Key features include district-level filtering across all 38 Tamil Nadu districts, date-range search, image and document evidence attachments, and an append-only correction mechanism backed by court order references - all enforced through an immutable blockchain ledger.

COMPLETED BY: SIVARAJ T